

# Important Topics to understand in Next.js

---

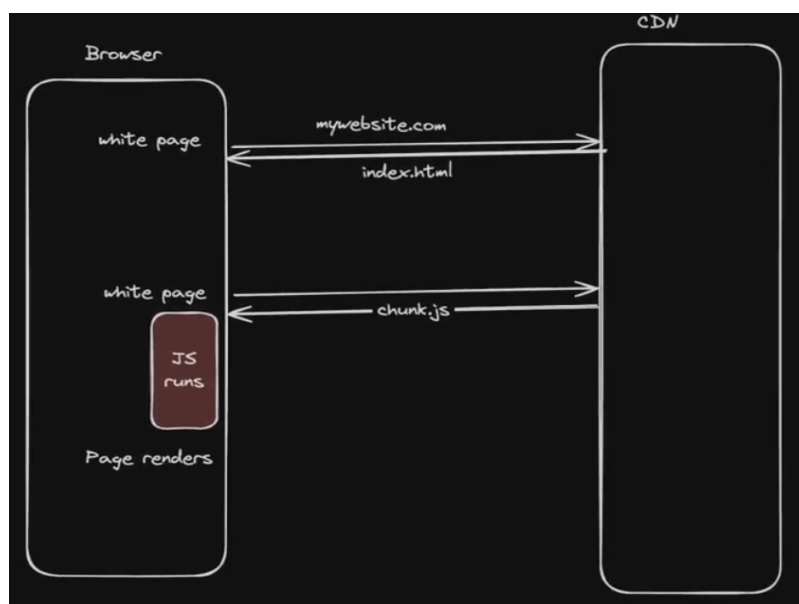
- Important Topics to understand in Next.js
  - Client side rendering (CSR)
    - Code flow for client side rendering
    - Downsides
  - Server side rendering (SSR)
    - Code flow for server side rendering
    - Why SSR ??
    - Downsides of SSR
  - Static site generation (SSG)
    - Why SSG ??
    - How to do SSG ??
    - Clearing the cache after some time
      - Another way to clear the cache
  - Assignment

## Client side rendering (CSR)

---

Client-side rendering (CSR) is a modern technique used in web development where the **rendering of a webpage is performed in the browser** using **JavaScript**. Instead of the server sending a fully rendered **HTML** page to the client

Good Example of CSR - **React** :-



## Code flow for client side rendering

---

**Step 1** -> You send a **request** to the website you want to visit

**Step 2** -> in **response**, you get an **EMPTY HTML page**[which has just `<script></script>` tag(consisting of the **JS** code file) and a **CSS** link tag generally]

**Step 3** -> Now you give another **request** to get the **JS** related code and now this **JS** code has its own **react** codebase and your own logic that you have written and this is what **runs on the page**

**Step 4** -> when the logic runs is **finally the page renders** which simply means that there is a slight time interval of **EMPTY page and after the js file runs then only you are able to see the contents of the website**

and hence this is why they are called as **Client side rendering** as **The HTML being injected into the DOM on the client**[Initially you were getting empty **HTML** and then you are filling it with the **JS** logic being run at later point of time]

Lets see a **react** project in action

- **Initialise a react project**

```
npm create vite@latest
```

- **Add dependencies**

```
npm i
```

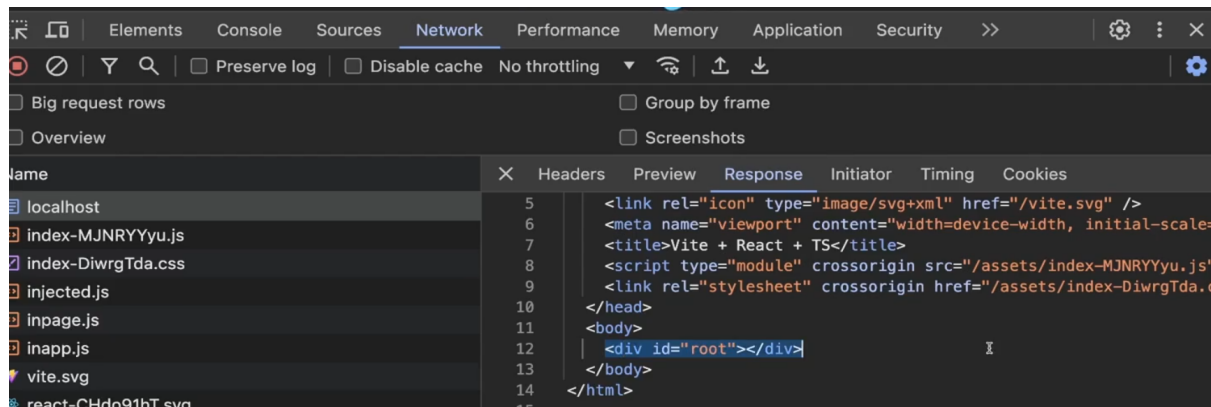
- **Start the project**

```
npm run build
```

- **Server the project**

```
cd dist/  
server
```

Open the network tab and notice how the initial **HTML** file didnt have any content(highlighted one) [you can also see the **script** tag and **css link** tag]

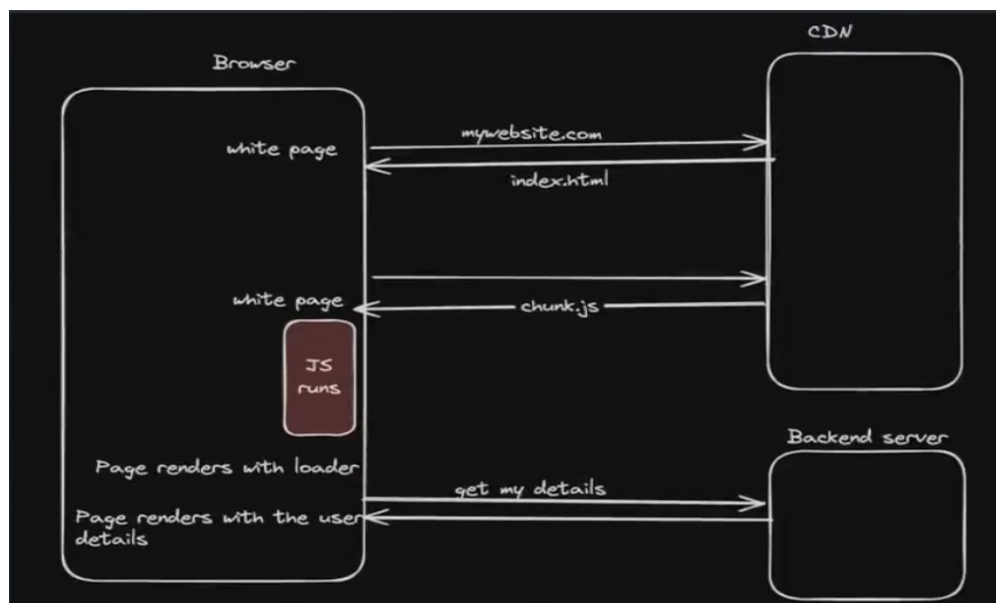


Eventually the **js** file which you see just below the **localhost** will come and hence populate the frontend (i.e. -> **Load the frontend**) **JS has the responsibility of rendering the things on the frontend**

**React** (or **CSR**) makes your life as a developer easy. You write components, **JS** renders them to the **DOM**.

## Downsides

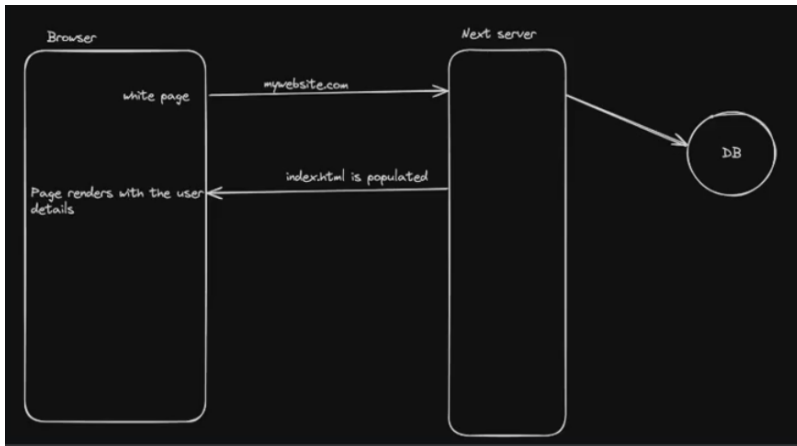
1. **Not SEO optimised**(google or any search engine **web crawlers** dont understand what your website does(as they only see at **HTML** file content) and hence your website does not able to display in the first search while searching for them in the search engine)
2. **User sees a flash before the page reloads**
3. **Waterfalling problem**



The above is the pic of Waterfalling problem

## Server side rendering (SSR)

When the **rendering** process (converting **JS** components to **HTML**) happens on the server, its called SSR.



## Code flow for server side rendering

**Step 1** -> In a `Next.js` application, you send the `request` to the backend(`Next.js` server).

**Step 2** -> Now `Next.js` server computes and then

**Step 3** -> gets all the data from the database(if required)

**Step 4** -> after getting all the data, **Server side renders all the things or basically page** and this **Readymade page is finally**

**Step 5** -> sent to the **frontend** and hence you see the website content

**So in the first request you were able to get most of the content of the website as response**

The very first `index.html` file you get as `response` will have rendering done because of the server side rendering

**Basically you get a PRE-RENDERED page**

Lets see `Next.js` project in action

- **Create `Next` app**

```
npx create-next-app
```

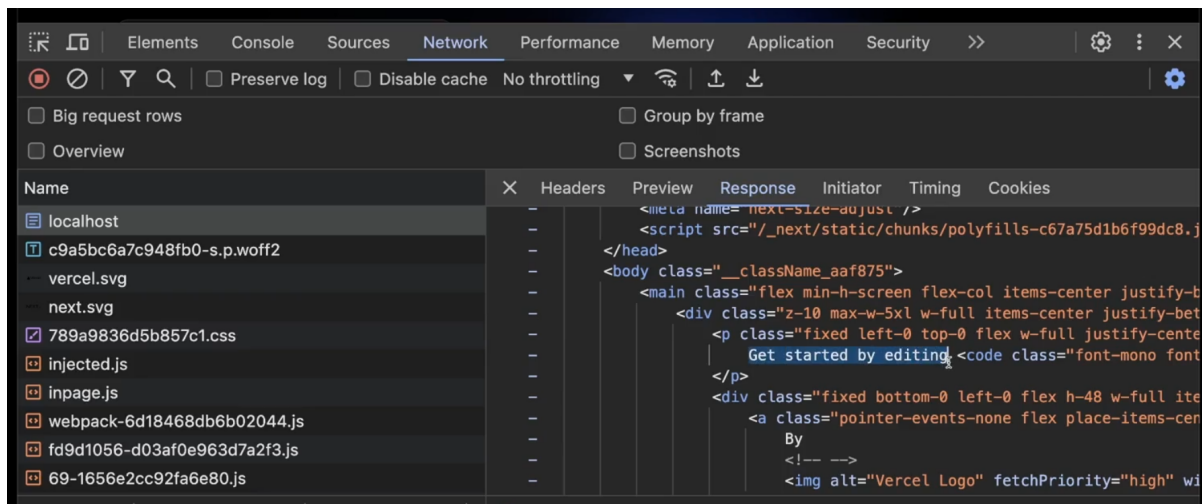
- **Build the project**

```
npm run build
```

- **Start the `NEXT` server**

```
npm run start
```

- Opening the Developer's tool



Notice how you have received **in the FIRST request only** the **HTML** page with all the things **being rendered**(it is now **NO MORE EMPTY FILE**) and hence this was **only possible due to SERVER-SIDE RENDERING**

**The backend itself populated the page, rendering happened over here and finally sent to the frontend**

## Why SSR ??

1. **SEO Optimisations**
2. **Gets rid of the waterfalling problem**(single **request** going on to get the desired output)
3. **No white flash before you see content**

## Downsides of SSR

1. **Expensive** since every request needs to render on the server
2. **Harder to scale, you can't cache to CDNs**

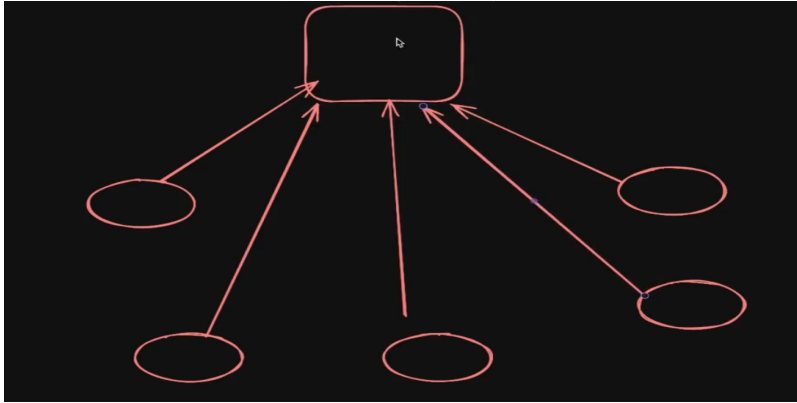
### Explanation of above written 1st point

When you were using **react.js**, then in all the machines, **rendering logic was running independently** something like the below :-



Circle represents the machine

But when using `Next.js`, then all the machines got **dependent on the `Next.js` server for rendering**



and the `Next.js` server is **doing the heavy operations or rendering FOR ALL THE MACHINES**

and **Rendering is an expensive operation(figuring out what to put on the DOM etc..)** and hence **COMPUTE INTENSIVE as well as EXPENSIVE**

You can make it **LESS EXPENSIVE** by using the Static site generation (SSG)

## Static site generation (SSG)

---

Reference to this topic -> [SSG in Next.js](#)

If a page uses **Static Generation**, the page HTML is generated at **build time**. That means in production, the page HTML is generated when you run `next build`. This **HTML** will then be reused on each request. It can be cached by a CDN.

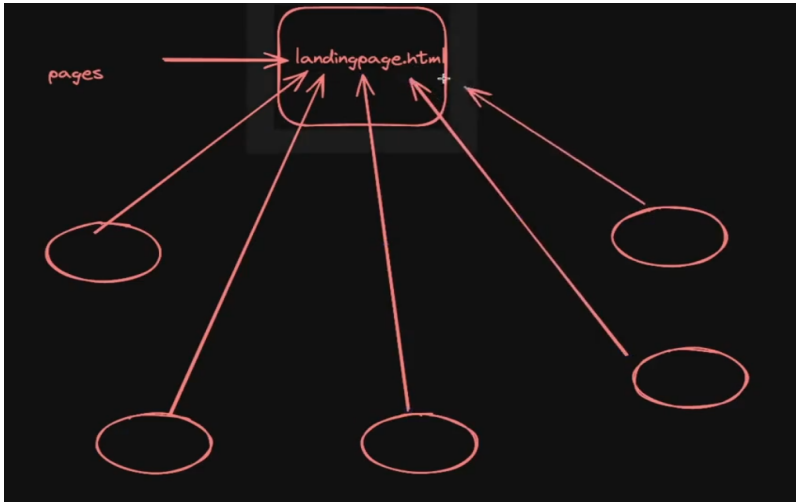
### Explation of the above para

You have your very expensive page(lets say your landing page) and right now you are using `Next.js`, as **many people ask you for the landing page the rendering happens on all of these pages and on the server you are able to do some of them BUT** What if i create a **landing page on the server and now anytime if someone asks for landing page i dont render them again i dont return them **HTML** again i just return them this premade landing page which is made on the server**

The above is what **making of static page means**

as the name suggest, **Dynamic page means every user is sending the **request** and corresponding to it, Server is generating or rendering the page for each **request** WHEREAS Static page means you create it once and every one is just going to fetch the same page**

Picture of Static site generation



If you know a certain page is going to be the same for everyone, it will look exact same for everyone, **WHY NOT RENDER IT ONCE during BUILD TIME**

💡 **What is build time ??**

-> Basically when you are running the command

```
npm run build
```

The time during which this command is running, **Why not PRE-GENERATE the page which is common to everyone and send it to everyone as Why do you want to keep RE-GENERATING the HTML page when you know it is going to be same for everyone.**

For ex -> if you go to [Project100x dev](#), the front page which you see is **Statically generated as everyone is going to see the same content**

Now if in the above page only, if you add a **progress bar for each user**, then it will be **DYNAMIC** and hence **Static page will not be possible to make**

**Just generate the page on the build time and if someone asks for that page, the server will not regenerate that page again, instead will return the same page which has been saved**

## Why SSG ??

---

If you use **static site generation**, you can defer the **expensive** operation of rendering a page to the **build time** so it only happens once.

## How to do SSG ??

---

Let's say you have an endpoint that gives you all the global todos of an app.

By **global todos** we mean that they are the same for all users, and hence this page can be statically generated.

here is the bunch of todos(backend endpoint) -> <https://sum-server.100xdevs.com/todos>

Assume every one has same set of todos that you see on the above link

Now given you have the above backend endpoint,

💡 **Create a statically generated page and put it in `Next.js` server**

Now if the static site generation concept is not present, then how you will achieve the above thing

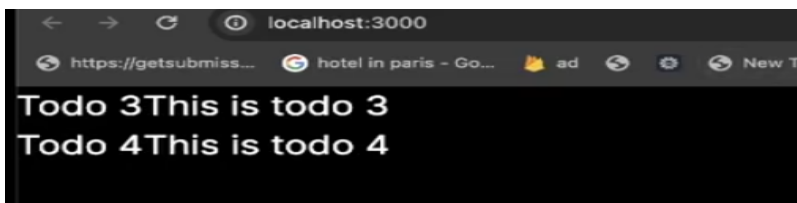
inside the `app`, file `page.tsx` the code will look something like this :-

```
export default async function Home(){
  const response = await fetch("https://sum-server.100xdevs.com/todos")
  const data = await response.json()

  return (
    <div>
      {data.todos.map((todo : any) => <div>
        {todo.title}
        {todo.description}
      </div>)}
    </div>
  )
}
```

You can also use `useEffect` as you know the two way of fetching the data from the backend (one is `useEffect` and other is making `async` component)

**output ->**



Now the backend point url you have used will be changing everytime you will reload the page of backend url, you will see the data change, but if you Notice the main page -> then the content will **NOT CHANGE (Reason -> it is statically generated)**

Now you dont have to use the conventional technique which we have read in the `Next.js` for how to add the static site generation, **`Next.js` is capable enough to understand which page can be made STATIC and hence makes them and also renders them according to the STATIC page concepts**

## Clearing the cache after some time

💡 **What if the `Next.js` server has some sensitive data that you needs to be cleared after some time ??**

-> The answer is that you **pass on some options along with data fetching from the backend**



adding the below line while data fetching from the backend will work for us :-

```
const res = await fetch('https://sum-server.100xdevs.com/todos', {
  next : {revalidate : 10} // 2
})
```

### Explanation of `// 2` code

Basically if there is a difference of 10 seconds between the two requests, then the second request will have to **re-fetch the response**

**Basically you INVALIDATE the cache by doing this. Very often used if you know that the data you are fetching is going to change TIME-to-TIME**

### Another way to clear the cache

making a folder structure like this (basically folder structure does not matter much, code do matter)

inside the `../lib/actions/action1`

```
'use server' // made server as the STATIC FILE is to be run on the server and so
do this

import { revalidateTag } from 'next/cache'

export default async function revalidate() { // 2
  revalidateTag('todos')
}
```

and then importing and using the `revalidate` on the main page

```
import revalidate from "../lib/actions/action1"; // first importing the revalidate
function from the made path folder

export default async function Home() {
  const response = await fetch('https://sum-server.100xdevs.com/todos', { next: {
tags: ['todos'] } })); // 3

  const data = await response.json();
  revalidate(); // 1

  return (
    <div>
      {data.todos.map((todo: any) => <div key={todo.id}>
        {todo.title}
        {todo.description}
      </div>)}
    </div>)}
  )
}
```

```
    </div>  
  );  
}
```

### Code flow of the above code

Anytime the `request` will come it will reach to the `// 1` line of code which will eventually lead to the `revalidate` function and its main work is to **anytime the request comes, it will REVALIDATE any place which has `todos` tag which the line of code `// 3` has and hence this line will again be revalidated**

**Now with every REFRESH, you will see new todos as the `revalidate()` function will run and hence it will do its work of RE-VALIDATION**

 **Now it really depends on you that where you want to use Static site or you even want to use it or not (OF COURSE TRY TO USE AS MUCH AS POSSIBLE due to the above said advantages)**

A good example of where static page are mostly of heavily used is ->

**BLOG websites(as you rarely change or update your blog which you once made) and thus you can make them static page**

## Assignment

---

 **How can you statically generate the contents of dynamic routes ??**

-> Google it and try to figure out how to achieve the above thing.